

MTH312 - HW1

Group 5

2026-01-24

Question: 1

Generate 500 observations from the standard normal distribution and draw a histogram based on this data. On the histogram, plot kernel density estimates based on the same data for various choices of well-known kernels. Compare the performance among the Histogram estimators and various kernel density estimators.

Solution

Kernel Density Estimation: Kernel Density Estimation (KDE) is a non-parametric method used to estimate the probability density function (PDF) of a random variable from observed data. It produces a smooth density curve without assuming any specific distribution, with the level of smoothness controlled by a bandwidth parameter. The functional form of Kernel Density Estimation (KDE) is

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - X_i}{h}\right),$$

where

- $X_1, X_2, \dots, X_n$ are the observed data points.
- $K(\cdot)$ is the kernel function (a symmetric probability density function such as the Gaussian kernel).
- $h > 0$ is the bandwidth (smoothing) parameter.
- n is the sample size.

Properties of KDE:

- **Non-negativity:**

$$\hat{f}(x) \geq 0 \quad \forall x \in X.$$

- **Normalization:** If $\int K(u) du = 1$, then

$$\int_{-\infty}^{\infty} \hat{f}(x) dx = 1.$$

- **Bias:** Under regularity conditions,

$$\text{Bias}[\hat{f}(x)] = \frac{h^2}{2} f''(x) \mu_2(K) + o(h^2),$$

where $\mu_2(K) = \int u^2 K(u) du$.

- **Variance:**

$$\text{Var}[\hat{f}(x)] = \frac{1}{nh} f(x) R(K) + o\left(\frac{1}{nh}\right),$$

where $R(K) = \int K^2(u) du$.

- **Mean Squared Error (MSE):**

$$\text{MSE}[\hat{f}(x)] = O(h^4) + O\left(\frac{1}{nh}\right).$$

- **Consistency:** If $h \rightarrow 0$ and $nh \rightarrow \infty$ as $n \rightarrow \infty$, then

$$\hat{f}(x) \xrightarrow{P} f(x).$$

Common Kernel Functions in KDE

- **Gaussian Kernel:**

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right),$$

which is smooth with infinite support and is widely used in practice.

- **Rectangular (Uniform) Kernel:**

$$K(u) = \begin{cases} \frac{1}{2}, & |u| \leq 1, \\ 0, & \text{otherwise,} \end{cases}$$

which assigns equal weight within a fixed interval.

- **Epanechnikov Kernel:**

$$K(u) = \begin{cases} \frac{3}{4}(1 - u^2), & |u| \leq 1, \\ 0, & \text{otherwise,} \end{cases}$$

which is optimal in a mean integrated squared error sense.

- **Biweight Kernel:**

$$K(u) = \begin{cases} \frac{15}{16}(1 - u^2)^2, & |u| \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

- **Triweight Kernel:**

$$K(u) = \begin{cases} \frac{35}{32}(1 - u^2)^3, & |u| \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Selection of Bandwidth

The bandwidth parameter h plays a crucial role in kernel density estimation, as it controls the smoothness of the estimated density function. An appropriate choice h balances the bias-variance trade-off.

- **Rule of Thumb (Silverman's Rule):**

$$h = 1.06 \hat{\sigma} n^{-1/5}.$$

- **IQR-based (Robust) Bandwidth Selection:**

$$h = 0.9 \min\left(\hat{\sigma}, \frac{\text{IQR}}{1.34}\right) n^{-1/5},$$

which is robust to outliers and skewness, and is commonly used for estimating the density function when the data deviates from normality.

- **Scott's Rule:**

$$h = \hat{\sigma} n^{-1/5}.$$

In our analysis, kernel density estimation is performed using an IQR-based bandwidth to ensure robustness against outliers.

```

n <- 500
sim.data <- rnorm(n, mean = 0, sd = 1)

# Kernel Functions
kern.gauss <- function(u) { (1 / sqrt(2 * pi)) * exp(-0.5 * u^2) }
kern.rect <- function(u) { ifelse(abs(u) <= 1, 0.5, 0) }
kern.tri <- function(u) { ifelse(abs(u) <= 1, 1 - abs(u), 0) }
kern.epan <- function(u) { ifelse(abs(u) <= 1, 0.75 * (1 - u^2), 0) }
kern.biweight <- function(u) { ifelse(abs(u) <= 1, (15/16) * (1 - u^2)^2, 0) }

# KDE Function
kde <- function(x, data, h, fn) {
  n <- length(data)
  est <- numeric(length(x))

  for (i in 1:length(x)) {
    u <- (x[i] - data) / h
    est[i] <- sum(fn(u)) / (n * h)
  }
  return(est)
}

# Bandwidth using Silverman Equation
iqr.val <- IQR(sim.data)
sigma.hat <- min(sd(sim.data), iqr.val / 1.34)
h <- 1.06 * sigma.hat * n^(-1/5)

# True density of data
true.dens.data <- dnorm(sim.data)

est.gauss.data <- kde(sim.data, sim.data, h, kern.gauss)
est.rect.data <- kde(sim.data, sim.data, h, kern.rect)
est.tri.data <- kde(sim.data, sim.data, h, kern.tri)
est.epan.data <- kde(sim.data, sim.data, h, kern.epan)
est.biwt.data <- kde(sim.data, sim.data, h, kern.biweight)

# Plotting
x.grid <- seq(-4, 4, length.out = 200)
est.gauss.grid <- kde(x.grid, sim.data, h, kern.gauss)
est.biwt.grid <- kde(x.grid, sim.data, h, kern.biweight)
est.rect.grid <- kde(x.grid, sim.data, h, kern.rect)
est.tri.grid <- kde(x.grid, sim.data, h, kern.tri)
est.epan.grid <- kde(x.grid, sim.data, h, kern.epan)
true.dens.grid <- dnorm(x.grid) # True density for plotting

```

Plot of different KDEs compared with the true distribution

```

par(mfrow = c(2, 3), mar = c(4, 4, 3, 1))

plot_kde <- function(title, est, col, lty = 1) {
  hist(

```

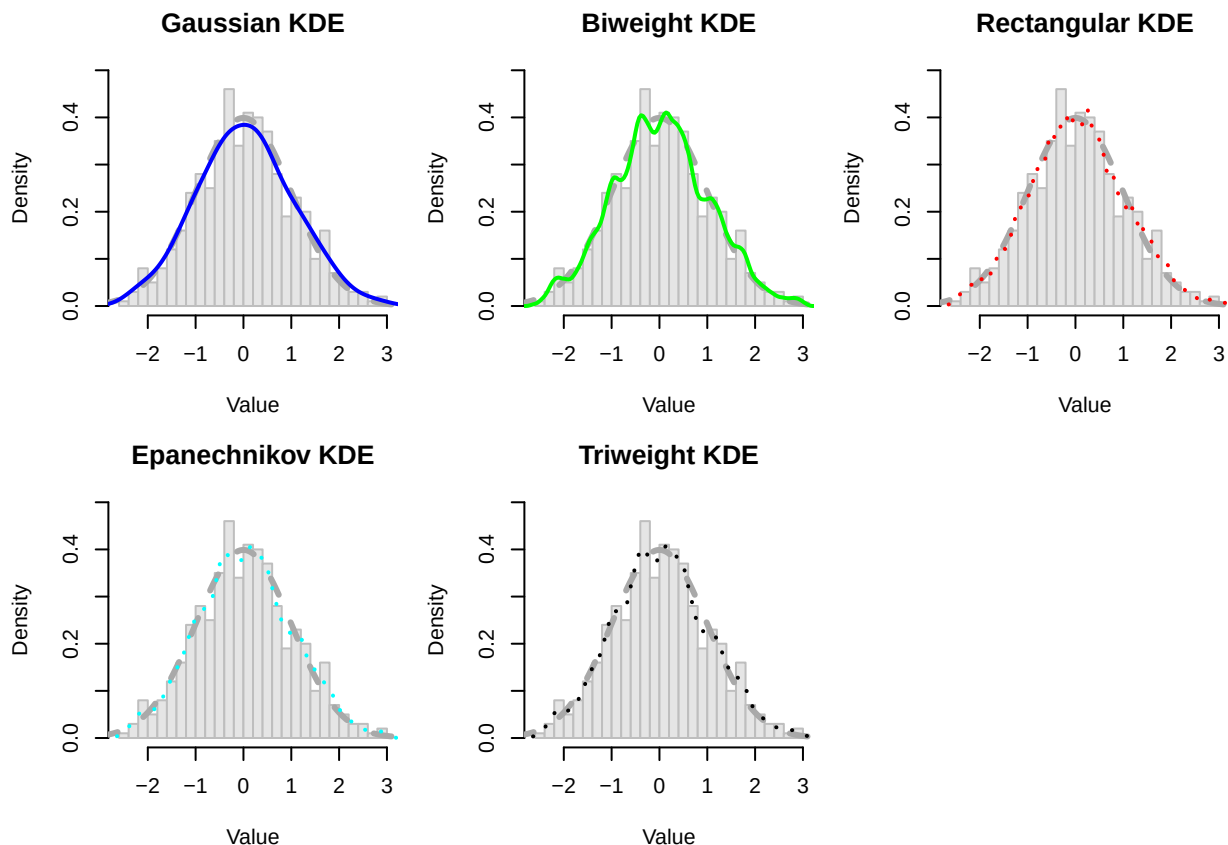
```

sim.data,
freq = FALSE,
breaks = 30,
col = "gray90",
border = "gray",
main = title,
xlab = "Value",
ylim = c(0, 0.5)
)
lines(x.grid, true.dens.grid, col = "darkgrey", lwd = 3, lty = 2)
lines(x.grid, est, col = col, lwd = 2, lty = lty)
}

plot_kde("Gaussian KDE",      est.gauss.grid, "blue")
plot_kde("Biweight KDE",     est.biwt.grid,  "green")
plot_kde("Rectangular KDE",  est.rect.grid,  "red",    3)
plot_kde("Epanechnikov KDE", est.epan.grid,  "#00FFFF", 3)
plot_kde("Triweight KDE",    est.tri.grid,   "black",   3)

par(mfrow = c(1, 1)) # reset

```



This plot represents a comparison of Kernel Density Estimates (KDEs) obtained using different kernel functions, all applied to the same dataset.

What each element shows

- **Histogram (grey bars):** Represents the empirical distribution of the observed data.
- **True Density (grey dashed line):** Denotes the actual underlying probability density function from which the data were generated and is used as a benchmark.
- **Colored curves (KDEs):** Show kernel density estimates computed using different kernel functions with the same bandwidth:
 - **Gaussian kernel** – smooth estimate with infinite support.
 - **Biweight kernel** – smoother estimate with compact support.
 - **Rectangular (Uniform) kernel** – rougher, piecewise constant estimate.
 - **Epanechnikov kernel** – optimal in the mean integrated squared error (MISE) sense.
 - **Triweight kernel** – very smooth estimate using a higher-order kernel.

We now evaluate the performance of different kernel functions using the Integrated Squared Error (ISE) and the Mean Squared Error (MSE) criteria.

The Integrated Squared Error (ISE) measures the overall discrepancy between the estimated density and the true density over the entire support and is defined as

$$\text{ISE} = \int_{-\infty}^{\infty} \left(\hat{f}(x) - f(x) \right)^2 dx,$$

where $\hat{f}(x)$ denotes the kernel density estimator and $f(x)$ denotes the true density function.

The Mean Squared Error (MSE) measures the local accuracy of the estimator at a point x and is given by

$$\text{MSE}(x) = \mathbb{E} \left[\left(\hat{f}(x) - f(x) \right)^2 \right].$$

In this context, smaller values of ISE and MSE indicate better performance of the kernel density estimator.

```
# Histogram Estimation for MSE and ISE
hist.obj <- hist(sim.data, breaks = 30, plot = FALSE)
bin.indices <- findInterval(sim.data, hist.obj$breaks, all.inside = TRUE)
est.hist.data <- hist.obj$density[bin.indices]

grid.bin.indices <- findInterval(x.grid, hist.obj$breaks)
est.hist.grid <- numeric(length(x.grid))
n_bins <- length(hist.obj$density)
valid.idx <- grid.bin.indices > 0 & grid.bin.indices <= n_bins
est.hist.grid[valid.idx] <- hist.obj$density[grid.bin.indices[valid.idx]]

# MSE Calculations & Comparison
mse.hist <- mean((est.hist.data - true.dens.data)^2) # Histogram MSE
mse.gauss <- mean((est.gauss.data - true.dens.data)^2)
mse.rect <- mean((est.rect.data - true.dens.data)^2)
mse.tri <- mean((est.tri.data - true.dens.data)^2)
mse.epan <- mean((est.epan.data - true.dens.data)^2)
mse.biwt <- mean((est.biwt.data - true.dens.data)^2)

# Integrated Squared Error (ISE) Calculation
dx <- x.grid[2] - x.grid[1]

# Helper function for ISE
calc_ise <- function(est, true, dx) {
  return(sum((est - true)^2) * dx)
```

```

}

ise.hist <- calc_ise(est.hist.grid, true.dens.grid, dx)
ise.gauss <- calc_ise(est.gauss.grid, true.dens.grid, dx)
ise.rect <- calc_ise(est.rect.grid, true.dens.grid, dx)
ise.tri <- calc_ise(est.tri.grid, true.dens.grid, dx)
ise.epan <- calc_ise(est.epan.grid, true.dens.grid, dx)
ise.biwt <- calc_ise(est.biwt.grid, true.dens.grid, dx)

# Comparison table
results <- data.frame(
  Estimator = c("Histogram", "Rectangular", "Triangular", "Epanechnikov", "Biweight", "Gaussian"),
  MSE = c(mse.hist, mse.rect, mse.tri, mse.epan, mse.biwt, mse.gauss),
  ISE = c(ise.hist, ise.rect, ise.tri, ise.epan, ise.biwt, ise.gauss)
)
results <- results[order(results$MSE), ] # Sort by best performance (lowest MSE)

cat("\n--- Performance Comparison (MSE) ---\n")

##
## --- Performance Comparison (MSE) ---

print(results, row.names = FALSE)

```

```

##      Estimator      MSE      ISE
##      Gaussian 0.0001554417 0.0007458099
##      Rectangular 0.0001690307 0.0009617031
##      Epanechnikov 0.0002485515 0.0011833434
##      Triangular 0.0003275360 0.0014826323
##      Biweight 0.0003833494 0.0016993863
##      Histogram 0.0016909508 0.0065723180

```

From the performance table, it can be concluded that the Gaussian kernel density estimator outperforms the other kernel estimators as well as the histogram method. This is evident from the fact that the Gaussian KDE achieves the lowest values of both Mean Squared Error (MSE) and Integrated Squared Error (ISE), indicating superior overall and local estimation accuracy.

Bootstrap-Based Comparison of KDE and Histogram

To compare the performance of kernel density estimators with the histogram, we employ a bootstrap-based approach.

Bootstrap Procedure

In each bootstrap replication, a sample is drawn with replacement from the original dataset. For every resampled dataset, density estimates are obtained using different kernel functions as well as the histogram estimator.

Calculation of ISE

For each bootstrap sample, the Integrated Squared Error (ISE) is computed for all estimators. A kernel density estimator is considered to outperform the histogram if its ISE value is smaller than that of the histogram for the same bootstrap sample.

By repeating this procedure over a large number of bootstrap replications, we count the number of times each KDE performs better than the histogram. This frequency provides an empirical measure of the relative performance of each kernel density estimator compared to the histogram.

```
# Bootstrap Procedure to Compare Histogram vs KDE
B = 1000 # Number of Bootstrap Samples
bootstrap.kernel <- function(sim.data, kernel, B, m = 200, grid.min = -4, grid.max = 4) {

  n <- length(sim.data)

  x.grid <- seq(grid.min, grid.max, length.out = m)
  f.true <- dnorm(x.grid)

  ISE.hist <- numeric(B)
  ISE.kde <- numeric(B)

  for (b in 1:B) {

    # Bootstrap resample
    x.boot <- sample(sim.data, size = n, replace = TRUE)

    # Histogram (Freedman-Diaconis)
    h.fd <- 2 * IQR(x.boot) / (n^(1/3))
    breaks <- seq(grid.min, grid.max, by = h.fd)

    hist.est <- hist(x.boot, breaks = breaks, plot = FALSE)

    f.hist <- approx(hist.est$mids, hist.est$density,
                     xout = x.grid,
                     rule = 1, yleft = 0, yright = 0)$y

    # Kernel Density Estimator
    f.kde <- kde(x.grid, x.boot, h, kernel)

    # Integrated Squared Error
    ISE.hist[b] <- mean((f.hist - f.true)^2)
    ISE.kde[b] <- mean((f.kde - f.true)^2)
  }

  list(
    kernel = kernel,
    kde.better.count = sum(ISE.kde < ISE.hist),
    ISE.hist = ISE.hist,
    ISE.kde = ISE.kde
  )
}

res.gaussian <- bootstrap.kernel(sim.data, kern.gauss, B)
res.epanechnikov <- bootstrap.kernel(sim.data, kern.epan, B)
```

```
res.uniform <- bootstrap.kernel(sim.data, kern.rect, B)
res.triangular <- bootstrap.kernel(sim.data, kern.tri, B)
res.biweight <- bootstrap.kernel(sim.data, kern.biweight, B)
```

```
cat("Gaussian KDE better than Histogram:",
    res.gaussian$kde.better.count, "times out of", B, "\n")
```

```
## Gaussian KDE better than Histogram: 982 times out of 1000
```

```
cat("Epanechnikov KDE better than Histogram:",
    res.epanechnikov$kde.better.count, "times out of", B, "\n")
```

```
## Epanechnikov KDE better than Histogram: 601 times out of 1000
```

```
cat("Rectangular KDE better than Histogram:",
    res.uniform$kde.better.count, "times out of", B, "\n")
```

```
## Rectangular KDE better than Histogram: 837 times out of 1000
```

```
cat("Triangular KDE better than Histogram:",
    res.triangular$kde.better.count, "times out of", B, "\n")
```

```
## Triangular KDE better than Histogram: 363 times out of 1000
```

```
cat("Biweight KDE better than Histogram:",
    res.biweight$kde.better.count, "times out of", B, "\n")
```

```
## Biweight KDE better than Histogram: 243 times out of 1000
```

```
# Boxplots of ISE Comparisons
```

```
par(mfrow = c(2, 3))
```

```
boxplot(res.gaussian$ISE.hist, res.gaussian$ISE.kde,
        names = c("Histogram", "Gaussian KDE"),
        col = c("gray", "lightblue"),
        main = "Bootstrap ISE Comparison")
```

```
legend("topright",
       legend = c("Histogram", "Gaussian KDE"),
       fill = c("gray", "lightblue"),
       bty = "n",
       cex = 0.5)
```

```
boxplot(res.epanechnikov$ISE.hist, res.epanechnikov$ISE.kde,
        names = c("Histogram", "Epanechnikov KDE"),
        col = c("gray", "lightblue"),
        main = "Bootstrap ISE Comparison")
```

```
legend("topright",
       legend = c("Histogram", "Epanechnikov KDE"),
       fill = c("gray", "lightblue"),
       bty = "n",
```



```

cex = 0.5)

boxplot(res.uniform$ISE.hist, res.uniform$ISE.kde,
        names = c("Histogram", "Rectangular KDE"),
        col = c("gray", "lightblue"),
        main = "Bootstrap ISE Comparison")

legend("topright",
       legend = c("Histogram", "Rectangular KDE"),
       fill = c("gray", "lightblue"),
       bty = "n",
       cex = 0.5)

boxplot(res.triangular$ISE.hist, res.triangular$ISE.kde,
        names = c("Histogram", "Triangular KDE"),
        col = c("gray", "lightblue"),
        main = "Bootstrap ISE Comparison")

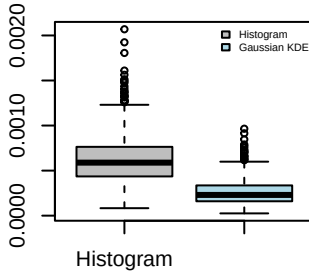
legend("topright",
       legend = c("Histogram", "Triangular KDE"),
       fill = c("gray", "lightblue"),
       bty = "n",
       cex = 0.5)

boxplot(res.biweight$ISE.hist, res.biweight$ISE.kde,
        names = c("Histogram", "Biweight KDE"),
        col = c("gray", "lightblue"),
        main = "Bootstrap ISE Comparison")

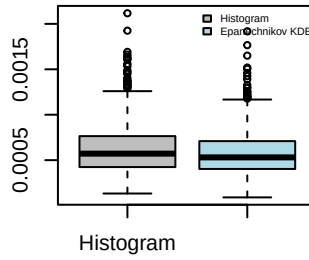
legend("topright",
       legend = c("Histogram", "Biweight KDE"),
       fill = c("gray", "lightblue"),
       bty = "n",
       cex = 0.5)

```

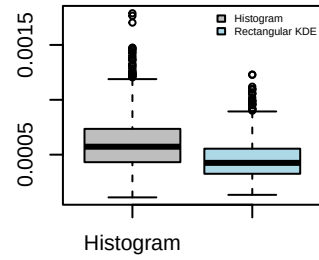
Bootstrap ISE Comparison



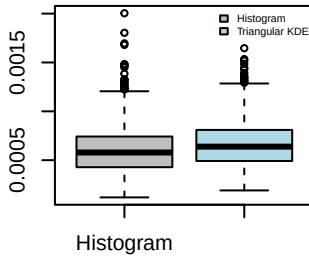
Bootstrap ISE Comparison



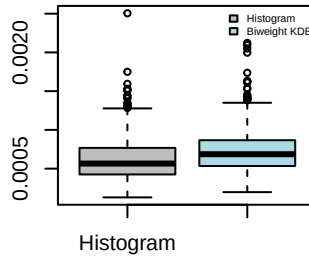
Bootstrap ISE Comparison



Bootstrap ISE Comparison



Bootstrap ISE Comparison



We clearly see that **Gaussian KDE** outperforms the histogram most of the time and performs better than the other KDE methods.

Question: 2

Generate 100 observations (Y_i, X_i) for $i = 1, \dots, 100$ from the model

$$Y = e^X + \varepsilon,$$

where $X \sim \text{Uniform}(0, 1)$ and

$$\mathbb{E}(\varepsilon \mid X) = 0 \quad \text{almost surely.}$$

Compare the performances of the Nadaraya–Watson estimator and the local linear estimator when the error term ε follows:

1. a standard normal distribution,
2. a standard Laplace distribution.

Solution

Nadaraya–Watson estimator (NW): The Nadaraya–Watson (NW) estimator is a nonparametric method used to estimate the regression function

$$m(x) = \mathbb{E}(Y \mid X = x).$$

Given observations $(X_1, Y_1), \dots, (X_n, Y_n)$, the NW estimator is defined as

$$\hat{m}_{NW}(x) = \frac{\sum_{i=1}^n K\left(\frac{x-X_i}{h}\right) Y_i}{\sum_{i=1}^n K\left(\frac{x-X_i}{h}\right)},$$

where $K(\cdot)$ is a kernel function and $h > 0$ is the bandwidth parameter.

Properties of the Nadaraya–Watson Estimator

- **Nonparametric:** The estimator does not assume any parametric form for the regression function $m(x)$.
- **Weighted Average:** The estimate $\hat{m}_{NW}(x)$ is a weighted average of the observed responses Y_i , with weights determined by the kernel function.
- **Bias:** The NW estimator suffers from boundary bias, especially near the edges of the support of X .
- **Variance:** The variance of the estimator depends on the bandwidth h ; smaller values of h lead to lower bias but higher variance.
- **Consistency:** Under suitable regularity conditions, if $h \rightarrow 0$ and $nh \rightarrow \infty$ as $n \rightarrow \infty$, then

$$\hat{m}_{NW}(x) \xrightarrow{P} m(x).$$

- **Sensitivity to Outliers:** The estimator can be sensitive to outliers, particularly when heavy-tailed error distributions are present.

Local Linear Estimator LL: The local linear estimator is a nonparametric regression method used to estimate the regression function

$$m(x) = \mathbb{E}(Y \mid X = x).$$

Given observations $(X_1, Y_1), \dots, (X_n, Y_n)$, the local linear estimator is obtained by solving

$$(\hat{a}, \hat{b}) = \arg \min_{a, b} \sum_{i=1}^n \{Y_i - a - b(X_i - x)\}^2 K\left(\frac{x - X_i}{h}\right),$$

where $K(\cdot)$ is a kernel function, and $h > 0$ is the bandwidth parameter. The local linear estimate of $m(x)$ is given by

$$\hat{m}_{LL}(x) = \hat{a}.$$

Properties of the Local Linear Estimator

- **Nonparametric:** The estimator does not assume any parametric form for the regression function.
- **Reduced Boundary Bias:** Compared to the Nadaraya–Watson estimator, the local linear estimator significantly reduces boundary bias.
- **Bias:** The local linear estimator has bias of order $O(h^2)$, similar to NW, but with improved behavior near boundaries.
- **Variance:** The variance is of order $O\left(\frac{1}{nh}\right)$ and depends on the bandwidth h .
- **Consistency:** Under standard regularity conditions, if $h \rightarrow 0$ and $nh \rightarrow \infty$ as $n \rightarrow \infty$, then

$$\hat{m}_{LL}(x) \xrightarrow{P} m(x).$$

- **Robustness:** The local linear estimator is generally more robust than the Nadaraya–Watson estimator, especially near the boundaries of the support.

Derivation of the Local Linear Estimator (Closed-Form Solution)

The local linear estimator (LLE) is derived by approximating the unknown regression function $m(x)$ locally around a target point x_0 using a first-order Taylor expansion. For observations (X_i, Y_i) sufficiently close to x_0 , we write

$$m(X_i) \approx \beta_0 + \beta_1(X_i - x_0),$$

where $\beta_0 \approx m(x_0)$ and $\beta_1 \approx m'(x_0)$.

Unlike global parametric regression, this approximation is only assumed to hold locally in a neighborhood of x_0 .

Local Weighted Least Squares Formulation

The coefficients (β_0, β_1) are estimated by minimizing a locally weighted sum of squared residuals:

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg \min_{\beta_0, \beta_1} \sum_{i=1}^n K\left(\frac{X_i - x_0}{h}\right) [Y_i - \beta_0 - \beta_1(X_i - x_0)]^2,$$

where:

- $K(\cdot)$ is a kernel function (Gaussian in this study),
- h is the bandwidth controlling the degree of localization.

This objective function assigns larger weights to observations closer to x_0 , enforcing locality.

Matrix Representation

Define the following quantities:

$$\mathbf{X} = \begin{pmatrix} 1 & X_1 - x_0 \\ 1 & X_2 - x_0 \\ \vdots & \vdots \\ 1 & X_n - x_0 \end{pmatrix}, \quad \mathbf{Y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix},$$
$$\mathbf{W} = \text{diag} \left(K\left(\frac{X_1 - x_0}{h}\right), \dots, K\left(\frac{X_n - x_0}{h}\right) \right).$$

The optimization problem can then be written compactly as

$$\min_{\boldsymbol{\beta}} (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{W} (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}), \quad \boldsymbol{\beta} = (\beta_0, \beta_1)^\top.$$

Closed-Form Solution

The objective function above is quadratic in β . Taking first-order conditions yields the weighted normal equations:

$$(\mathbf{X}^\top \mathbf{W} \mathbf{X}) \hat{\beta} = \mathbf{X}^\top \mathbf{W} \mathbf{Y}.$$

Provided that $\mathbf{X}^\top \mathbf{W} \mathbf{X}$ is invertible, the solution is given explicitly by

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W} \mathbf{Y}.$$

The local linear estimator of the regression function at the target point x_0 is

$$\hat{m}_{LL}(x_0) = \hat{\beta}_0.$$

Thus, although the estimator is defined through an optimization problem, its quadratic structure admits an exact closed-form solution at each evaluation point.

Connection to the R Implementation

The function `loc.lin.closed` in the script directly implements this closed-form solution:

- `X.mat <- cbind(1, X - x0)` constructs the design matrix $\mathbf{X}$.
- `W.mat <- diag(w)` constructs the weight matrix $\mathbf{W}$.
- `lhs <- t(X.mat) %*% W.mat %*% X.mat` computes $\mathbf{X}^\top \mathbf{W} \mathbf{X}$.
- `rhs <- t(X.mat) %*% W.mat %*% Y` computes $\mathbf{X}^\top \mathbf{W} \mathbf{Y}$.
- `solve(lhs, rhs)` solves the weighted normal equations.
- `beta.hat[1]` extracts $\hat{\beta}_0 = \hat{m}_{LL}(x_0)$.

Therefore, no numerical optimization routine is required: the estimator is evaluated by solving a 2×2 linear system at each target point.

Why Local Linear Improves Boundary Performance

The Nadaraya–Watson estimator effectively performs a locally weighted average and suffers from boundary bias due to asymmetric kernel support near the edges of the design space.

In contrast, the local linear estimator fits a local first-order polynomial, allowing it to adapt to such asymmetry. As a result, the leading bias term at the boundary vanishes, yielding superior boundary behavior without increasing variance order.

Summary

Although the local linear estimator is motivated by an optimization problem, its quadratic structure ensures a closed-form solution. This combination of theoretical tractability and improved finite-sample behavior explains its superior performance under regular error distributions, as observed in the simulation and bootstrap results.

```
# Data Generation
set.seed(123)
n <- 100
x <- sort(runif(n, 0, 1))
true.m <- exp(x)

# Errors
eps.norm <- rnorm(n, 0, 1)
u.unif <- runif(n, -0.5, 0.5)
eps.lap <- -sign(u.unif) * log(1 - 2 * abs(u.unif))

y.norm <- true.m + eps.norm
y.lap <- true.m + eps.lap

# Estimators
# Nadaraya-Watson
nadaraya.watson <- function(x, X, Y, h) {
  res <- numeric(length(x))
  for(i in 1:length(x)) {
    x0 <- x[i]
    w <- kern.gauss((X - x0) / h)
    res[i] <- sum(w * Y) / sum(w)
  }
  return(res)
}

# LLE Closed Form
loc.lin.closed <- function(x, X, Y, h) {
  res <- numeric(length(x))
  for(i in 1:length(x)) {
    x0 <- x[i]
    w <- kern.gauss((X - x0) / h)
    W.mat <- diag(w)
    X.mat <- cbind(1, X - x0)

    # Solve  $(X'WX)\beta = X'WY$ 
    XtW <- t(X.mat) %*% W.mat
    lhs <- XtW %*% X.mat
    rhs <- XtW %*% Y

    beta.hat <- tryCatch({ solve(lhs, rhs) }, error = function(e) c(NA, NA))
    res[i] <- beta.hat[1]
  }
  return(res)
}
```

```

# Execution
h <- 0.1
est.nw.norm <- nadaraya.watson(x, x, y.norm, h)
est.lle.norm <- loc.lin.closed(x, x, y.norm, h)
est.nw.lap <- nadaraya.watson(x, x, y.lap, h)
est.lle.lap <- loc.lin.closed(x, x, y.lap, h)

# MSE
mse.nw.norm <- mean((est.nw.norm - true.m)^2)
mse.lle.norm <- mean((est.lle.norm - true.m)^2)
mse.nw.lap <- mean((est.nw.lap - true.m)^2)
mse.lle.lap <- mean((est.lle.lap - true.m)^2)

```

Evaluation of NW and LLE under Normal and Laplace Errors

We evaluate the performance of the Nadaraya–Watson (NW) estimator and the local linear estimator (LLE) using the same set of design points. The response variable is generated under two different error distributions: the standard normal distribution and the double exponential (Laplace) distribution.

```

##
## --- Regression MSE Results ---

## NW (Normal):    0.03449617

## LLE (Normal):   0.02883775

## NW (Laplace):   0.04104945

## LLE (Laplace):  0.06087161

```

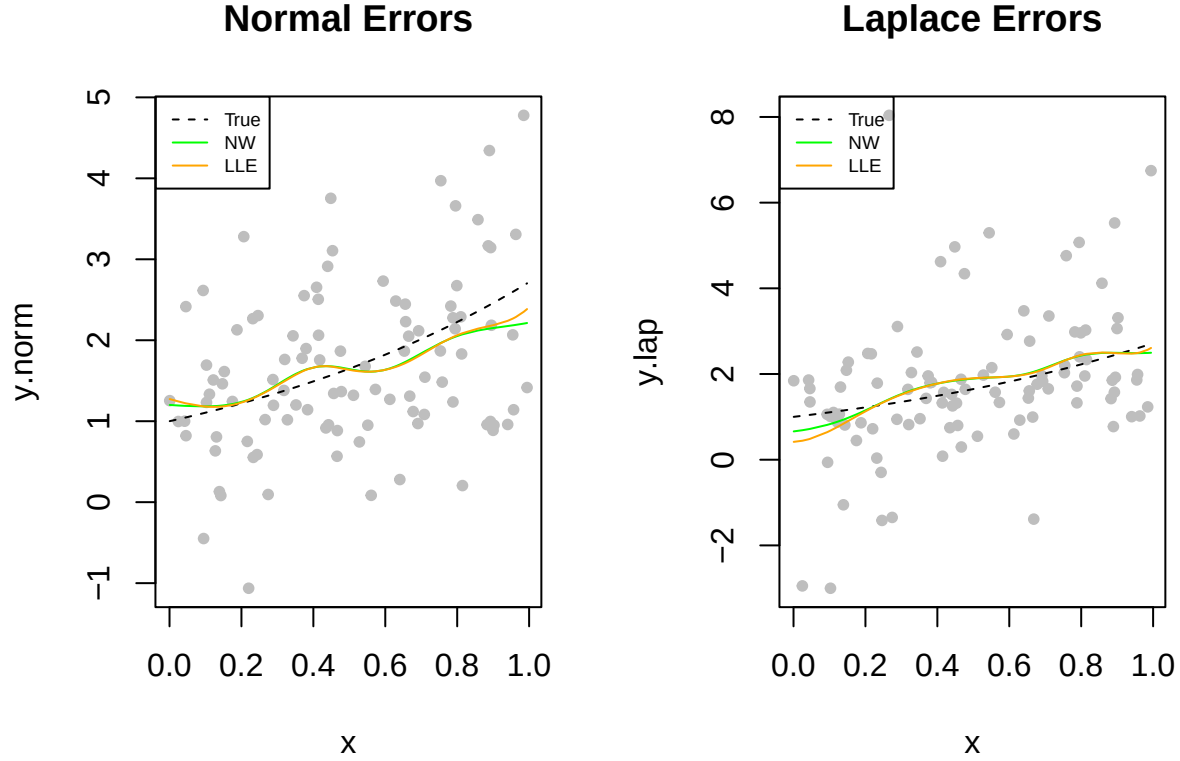
Plot

```

# Plotting
par(mfrow = c(1, 2))
plot(x, y.norm, pch = 20, col = "gray", main = "Normal Errors")
lines(x, true.m, lty = 2); lines(x, est.nw.norm, col = "green"); lines(x, est.lle.norm, col = "orange")
legend("topleft", legend = c("True", "NW", "LLE"), col = c("black", "green", "orange"),
      lty = c(2, 1, 1), cex = 0.6)

plot(x, y.lap, pch = 20, col = "gray", main = "Laplace Errors")
lines(x, true.m, lty = 2); lines(x, est.nw.lap, col = "green"); lines(x, est.lle.lap, col = "orange")
legend("topleft", legend = c("True", "NW", "LLE"), col = c("black", "green", "orange"),
      lty = c(2, 1, 1), cex = 0.6)

```



Normal Errors:

When the distribution of the random error is normal, the local linear estimator performs better compared to the Nadaraya–Watson estimator.

Laplace Errors:

When the distribution of the random error is Laplace (double exponential), the Nadaraya–Watson estimator performs better compared to the local linear estimator.

Bootstrap-Based Comparison of Nadaraya–Watson and Local Linear Estimators

To compare the performance of the Nadaraya–Watson (NW) estimator and the local linear method (LLM), we employ a bootstrap-based approach under different error distributions.

Bootstrap Procedure

In each bootstrap replication, a sample is drawn with replacement from the original dataset. For every resampled dataset, the regression function is estimated using both the Nadaraya–Watson estimator and the local linear estimator. The procedure is carried out separately for the cases where the error term follows a standard normal distribution and a double exponential (Laplace) distribution.

Calculation of MSE

For each bootstrap sample, the Mean Squared Error (MSE) is computed for both estimators. The MSE is evaluated by comparing the estimated regression function with the true regression function at a set of grid points.


```

bootstrap.compare <- function(X, Y, true.m, h, B = 1000) {

  n <- length(X)

  mse.nw <- numeric(B)
  mse.ll <- numeric(B)

  for (b in 1:B) {
    idx <- sample(1:n, size = n, replace = TRUE)

    Xb <- X[idx]
    Yb <- Y[idx]
    true.mb <- true.m[idx]

    # Sort (important for smoothness)
    ord <- order(Xb)
    Xb <- Xb[ord]
    Yb <- Yb[ord]
    true.mb <- true.mb[ord]

    est.nw <- nadaraya.watson(Xb, Yb, h)
    est.ll <- loc.lin.closed(Xb, Yb, h)

    mse.nw[b] <- mean((est.nw - true.mb)^2, na.rm = TRUE)
    mse.ll[b] <- mean((est.ll - true.mb)^2, na.rm = TRUE)
  }

  list(
    mse.nw = mse.nw,
    mse.ll = mse.ll,
    prob.ll.better = mean(mse.ll < mse.nw),
    prob.nw.better = mean(mse.nw < mse.ll)
  )
}

boot.norm <- bootstrap.compare(x, y.norm, true.m, h, B = 1000)
boot.lap <- bootstrap.compare(x, y.lap, true.m, h, B = 1000)

cat("\n--- Bootstrap Results (10000 replications) ---\n")

```

```

##
## --- Bootstrap Results (10000 replications) ---

```

```

cat("\nNormal errors:\n")

```

```

##
## Normal errors:

```

```

cat("P(LL better than NW): ", boot.norm$prob.ll.better, "\n")

```

```

## P(LL better than NW): 0.385

```

```
cat("P(NW better than LL): ", boot.norm$prob.nw.better, "\n")
```

```
## P(NW better than LL): 0.615
```

```
cat("\nLaplace errors:\n")
```

```
##
```

```
## Laplace errors:
```

```
cat("P(LL better than NW): ", boot.lap$prob.ll.better, "\n")
```

```
## P(LL better than NW): 0.063
```

```
cat("P(NW better than LL): ", boot.lap$prob.nw.better, "\n")
```

```
## P(NW better than LL): 0.937
```

```
par(mfrow = c(1, 2))
```

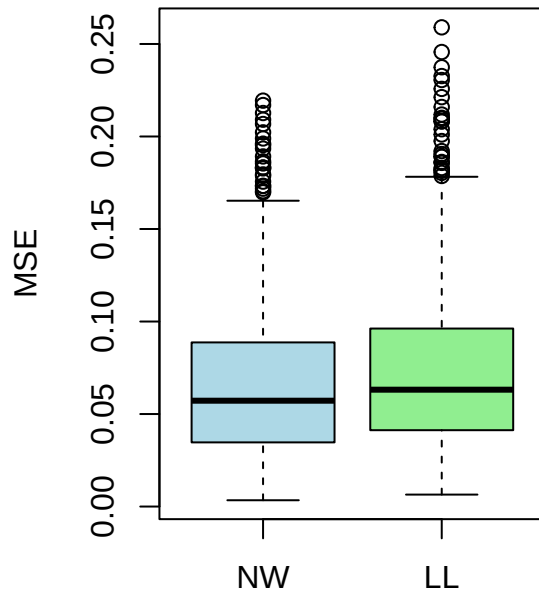
```
# Normal errors
```

```
boxplot(
  boot.norm$mse.nw,
  boot.norm$mse.ll,
  names = c("NW", "LL"),
  main = "MSE Comparison (Normal Errors)",
  ylab = "MSE",
  col = c("lightblue", "lightgreen")
)
```

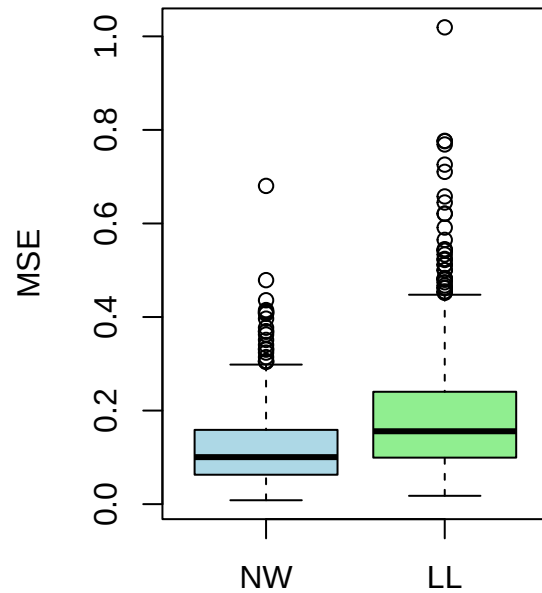
```
# Laplace errors
```

```
boxplot(
  boot.lap$mse.nw,
  boot.lap$mse.ll,
  names = c("NW", "LL"),
  main = "MSE Comparison (Laplace Errors)",
  ylab = "MSE",
  col = c("lightblue", "lightgreen")
)
```

MSE Comparison (Normal Errors:



MSE Comparison (Laplace Error:



In the bootstrap analysis, under normal error distribution, the probability that the local linear estimator (LLE) performs better than the Nadaraya–Watson (NW) estimator is approximately 0.615. This indicates that, out of 1000 bootstrap replications, the LLE outperforms the NW estimator in about 615 cases.

Under the Laplace (double exponential) error distribution, the probability that the Nadaraya–Watson estimator performs better than the local linear estimator is approximately 0.93. This implies that, out of 1000 bootstrap replications, the NW estimator outperforms the LLE in about 930 cases.